

Prompts & Design Documentation

Unified Enterprise AI Agent — Track 3

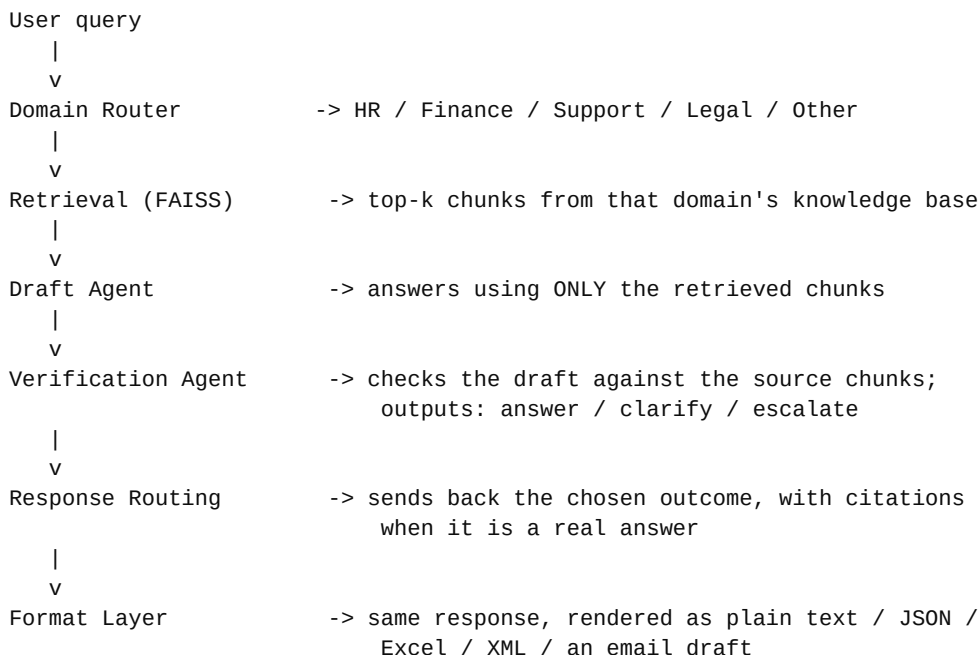
KOHLER-MITWPU AI Research Centre — Case Study Submission

Prepared by Raghav Uday Menon | B.Tech CSE (AI & DS), MIT-WPU

This document contains the exact prompts running in the deployed system, the reasoning behind the key design decisions in the pipeline, a short account of how the project was actually built and debugged, and a summary of the evaluation results. It is meant to be read alongside the GitHub repository and the live demo, not instead of them.

1. System Overview

The agent answers employee and customer questions across four domains — HR, Finance, Customer Support, and Legal/Compliance — by routing each query to a domain-specific knowledge base, retrieving the most relevant source chunks, drafting an answer strictly from that retrieved context, and then independently verifying the draft before it is shown to the user. Verification is what the rest of this document mostly explains: instead of always returning an answer with the same confident tone, the system decides between three outcomes — answer, ask a clarifying question, or escalate to a human team — based on whether the draft is actually grounded in the retrieved documents.



All three LLM calls (classify, draft, verify) currently run on Google's Gemini **gemini-3.5-flash-lite** model through the `google-genai` SDK. Embeddings for retrieval are computed locally with **sentence-transformers** (`all-MiniLM-L6-v2`) so no API call is needed for that step, and search runs over four separate **FAISS** indexes, one per domain.

2. System Prompts (Verbatim, as Deployed)

The three prompts below are copied directly from `src/agents.py` in the submitted repository — not an earlier draft. Each one was iterated on after being caught failing on a specific real test case; Section 3 explains what those failures were and how the fix is reflected in the wording below.

CLASSIFY_DOMAIN_SYSTEM_PROMPT

You are a routing classifier for an enterprise support assistant.

Classify the user's query into exactly one of the following domains:

- HR: employee benefits, leave, onboarding, performance reviews, workplace conduct, compensation
- FINANCE: expenses, budgets, invoices, procurement, payments, reimbursement, tax
- SUPPORT: product setup, troubleshooting, warranty, returns, device pairing
- LEGAL: contracts, compliance, data privacy, intellectual property, regulatory matters
- OTHER: anything that does not clearly fit HR, FINANCE, SUPPORT, or LEGAL

Respond with exactly one word: HR, FINANCE, SUPPORT, LEGAL, or OTHER.

Do not include punctuation, explanation, or any other text.

DRAFT_ANSWER_SYSTEM_PROMPT

You are an enterprise knowledge assistant. Answer the user's question using ONLY the information contained in the provided context chunks. Do not use outside knowledge and do not guess.

The conversation history may contain earlier turns in this exchange - use it to resolve follow-up questions (e.g. pronouns, "what about...", implied subjects) so your answer addresses what the user actually means.

If the context does not contain enough information to answer confidently, respond exactly with:

"I don't have enough information to answer confidently"

followed by a brief note on what information is missing, rather than guessing or inferring beyond the context.

If the answer depends on a variable (such as role, contract type, or category) and the context enumerates the specific values for each case, answer by listing all the relevant cases rather than asking the user to specify or declining to answer. Only treat a question as needing clarification when the missing information is about the user's own specific situation (e.g. which product they own, their exact travel destination, their purchase date) and is not itself enumerable from the context.

When you do answer, be concise. Write in clean prose without bracketed citations, chunk numbers, or other inline reference markers - do not mention "Chunk" or similar labels in your answer.

VERIFY_ANSWER_SYSTEM_PROMPT

You are a quality-control reviewer for an enterprise knowledge assistant. You will be given a draft answer and the source context chunks it was supposed to be based on.

Evaluate whether the draft is fully grounded in the source chunks (no unsupported claims, no facts absent from the context) and decide how confident and actionable the draft is.

Important rule: if the draft itself states that it doesn't have enough information, cannot answer confidently, or otherwise contains no substantive answer to the question, then action MUST be "escalate" - regardless of how well-reasoned or appropriate that refusal is. The "answer" action is only for drafts that provide an actual grounded answer to the question, never for drafts that correctly decline to answer. A

well-written refusal is still not a confident answer.

"grounded" must be true only when action is "answer" and the draft provides a real, evidence-backed response supported by the source chunks. In every other case - "escalate" (whether because the draft refused to answer or because it contained ungrounded content) or "clarify" - "grounded" must be false, since there is no substantive answer being made that could be evaluated as grounded.

Respond with ONLY a single valid JSON object (no markdown formatting, no surrounding text) with exactly these fields:

```
{
  "grounded": <true only if action is "answer" and the draft is a real answer fully
    supported by the source chunks; false in every other case, including
    "escalate" and "clarify">,
  "confidence": <"high", "medium", or "low">,
  "action": <"answer" if the draft provides an actual grounded answer and is ready to
    send as-is, "clarify" if the user's question needs clarification before a
    confident answer is possible, or "escalate" if this should go to a human -
    including whenever the draft itself declines to answer or says it lacks
    enough information>,
  "reasoning": <a brief string explaining the evaluation>
}
```

3. Design Rationale

Each of these decisions came out of a specific failure the eval harness caught, not from guessing upfront what might go wrong. That is deliberate — the prompts above are the result of testing against 20 hand-written questions and fixing what actually broke.

3.1 Why verification is a separate call, not part of drafting

An earlier version asked the drafting model to also self-report its own confidence in the same response. In practice, a model grading its own work tends to rate itself favorably. Splitting verification into a second, independent call — one that only ever sees the draft and the source chunks, never the instruction to "be helpful" that shaped the draft — gives the review a genuinely separate vantage point, closer to how a second reviewer would check a colleague's work rather than the same person re-reading their own answer.

3.2 The escalate-on-refusal rule

The most important bug caught during evaluation: when the draft agent correctly said *"I don't have enough information to answer confidently,"* the verification agent was originally rating that refusal as `grounded: true, confidence: high, action: answer` — because, taken on its own, the refusal text was an accurate, well-supported statement. The system would then have shipped a non-answer to the user labelled as a confident one, which defeats the entire point of having a verification layer. The fix is the explicit rule in the VERIFY prompt: a refusal is never an "answer," no matter how well-reasoned it is, and must always route to `escalate`. This was verified by rerunning the exact test case that exposed it and confirming the action flipped to `escalate`.

3.3 The grounded/action consistency rule

A follow-on issue after the fix above: `grounded: true` could still appear alongside `action: escalate`, which is a confusing state to log or show a reviewer — "grounded" is supposed to mean "safe to trust as-is." The prompt now states directly that `grounded` must be false whenever `action` is not "answer", so the two fields can never contradict each other downstream.

3.4 Enumerate, don't ask, when the context already lists every case

Some questions look like they need clarification ("what's the approval limit for purchases?") but the knowledge base actually enumerates the answer for every role already. Early on, the draft agent asked "which role are you?" instead of just listing all of them — technically safe, but worse for the user than answering directly. The prompt now distinguishes: if the context enumerates every case of a variable, list them all; only ask for clarification when the missing piece is about the user's own situation and isn't something the context could enumerate anyway (which product they own, their exact date of travel). Two test cases (purchase-approval limits, contract review turnaround time) were manually reclassified from `clarify` to `answer` in the eval set once this rule was in place, since a correct system should now answer them directly.

3.5 Citation relevance threshold

Retrieval returns the top-k chunks for a query, but not every returned chunk is necessarily relevant enough to show the user as a citation — a chunk can rank in the top few purely by keyword overlap without actually supporting the answer. A similarity-score cutoff (raised from 0.38 to 0.40 after spot-checking citation quality) is applied only to what's shown to the user as a source citation. It is deliberately not applied to what's fed into drafting and verification, because multi-hop questions sometimes need a chunk that scores lower on raw

similarity but is still necessary to combine with a higher-scoring one for a complete answer.

3.6 Model consolidation onto Flash-Lite

The original design used a stronger model for the drafting step and Flash-Lite for the cheaper classify/verify steps. That plan met a hard constraint almost immediately: the stronger tier's free daily quota is 20 requests a day, which a single 20-question eval run burns through by itself. Flash-Lite's quota is far more generous, so drafting was moved onto it as well — all three calls now share one model. This is a real engineering decision made after hitting the constraint in practice, not a compromise hidden after the fact, and the architecture would not need to change to move back to a stronger model on a paid plan.

3.7 Client-side rate limiting

Consolidating onto one model then exposed its per-minute limit (15 requests/minute on the free tier): a single query can trigger up to three calls internally (classify, draft, verify), so a short burst of user activity could exceed the limit even with the daily quota untouched. A small client-side limiter in `agent.s.py` tracks the timestamp of the last Gemini call and sleeps as needed to keep calls at least ~4.2 seconds apart, regardless of which function triggers them. This is also why the deployed app takes 10-15 seconds to respond — it is intentional pacing, not a performance problem.

4. Development Workflow

The system was built iteratively inside VS Code: defining the architecture and scope up front, writing the knowledge-base content and prompts directly, and then building out the source files (retrieval pipeline, agent calls, Streamlit UI, evaluation harness) against those specifications. Every fix described in Section 3 followed the same loop:

- Run the 20-question evaluation harness (`eval/run_eval.py`) and look at exactly which cases disagreed with the expected outcome, not just the pass rate.
- Read the actual reasoning field the verification agent produced for the failing case, to see *why* it made that call rather than guessing.
- Change one prompt rule at a time to address that specific failure, rerun only the affected test case first to confirm the fix, then rerun the full set to check nothing else regressed.

Two examples worth calling out specifically, since they were the clearest instances of a prompt bug rather than a genuinely ambiguous case: the refusal-marked-as-answer bug (Section 3.2), caught on a question about stock option vesting that isn't covered anywhere in the knowledge base by design — the draft correctly said it didn't know, but verification was initially about to ship that as a confident answer; and the purchase-approval enumeration bug (Section 3.4), caught on a question where the knowledge base already listed every approval-limit tier by role, but the draft asked the user to specify their role instead of just listing them.

The project also went through a budget-driven architecture pivot partway through: it was originally built against the Anthropic API, but that requires a funded billing account with no meaningful free tier, which wasn't an option here. The entire backend was moved to Google's Gemini API instead (genuinely free, no card required), which took a rewrite of the agent-calling code but no change to the pipeline architecture itself, since the interface between stages (retrieve → draft → verify → route) never depended on which model was making the call.

The finished system is deployed on Streamlit Community Cloud so it can be evaluated without any local setup: `kohler-enterprise-agent.streamlit.app`.

5. Evaluation Summary

`eval/test_set.json` contains 20 hand-written test questions spanning four categories: straightforward questions answerable from a single document, multi-hop questions that require combining two documents, genuinely ambiguous questions, and questions on topics deliberately left out of the knowledge base entirely (to test that the system escalates rather than fabricates an answer).

Metric	Result
Test cases	20
Passed (matched expected action)	15 / 20 (75%)
Hallucinated or fabricated facts observed	0, across every run performed
Cause of every miss	Boundary disagreement on genuinely ambiguous questions — answer / clarify / escalate

The number reported here (75%) is the honest result after the fixes in Section 3 were applied — it was not tuned further to chase a higher score once every miss was confirmed to be a genuine boundary-judgment

disagreement rather than a bug. What matters more for an enterprise assistant than a slightly higher pass rate on fuzzy cases is the zero-hallucination result: in every evaluation run performed during development, the system never stated a fact that was not present in its retrieved source documents. When it did not know something, it said so, consistently, rather than guessing.

Known limitations

- Response latency is 10-15 seconds per answer, a direct consequence of the client-side rate limiting described in Section 3.7 rather than a modeling limitation.
- The answer / clarify / escalate decision is not perfectly consistent on genuinely ambiguous questions — asking the same borderline question twice can produce two different but equally reasonable outcomes. This reflects the underlying uncertainty in those specific questions, not an implementation defect, and is not something that can be fully eliminated without removing the ambiguity from the question itself.